

Sockets: la base de toda comunicación en red

Lo que ya usas sin saberlo

Todo lo que has hecho hasta ahora en este módulo —peticiones REST, JWT, RabbitMQ— viaja, por debajo, sobre una pieza que nunca has tocado directamente: el socket.

Este apartado saca el socket a la superficie: qué es, quién lo abre, y cómo viajan los datos por él.

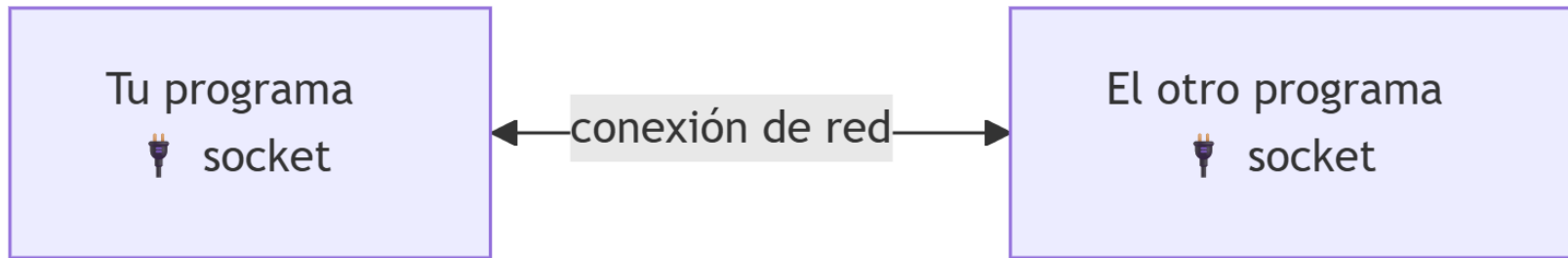
Qué es un socket: IP + puerto

Dos programas que se comunican por red lo hacen a través de una conexión: un canal identificado por dos datos, por ejemplo en 192.168.1.5:8080.

192.168.1.5	8080
IP — qué máquina, de entre todas las de la red	Puerto — qué programa concreto, de entre todos los que corren en esa máquina

La IP es el portal del edificio; el puerto es el piso concreto dentro de ese portal. Un socket es el objeto de tu código que representa uno de los dos extremos de esa conexión.

Cada programa tiene el suyo, en su propio lado



```
netstat -an | grep 8080    # LISTEN -> el socket que espera
netstat -an | grep 5432    # ESTABLISHED -> ya abierto con PostgreSQL
```

Cuando curl habla con tu aplicación en el 8080, o tu aplicación se conecta a PostgreSQL (5432), MongoDB (27017) o RabbitMQ (5672), hay un socket TCP a cada lado en todos esos casos.

Roles cliente y servidor

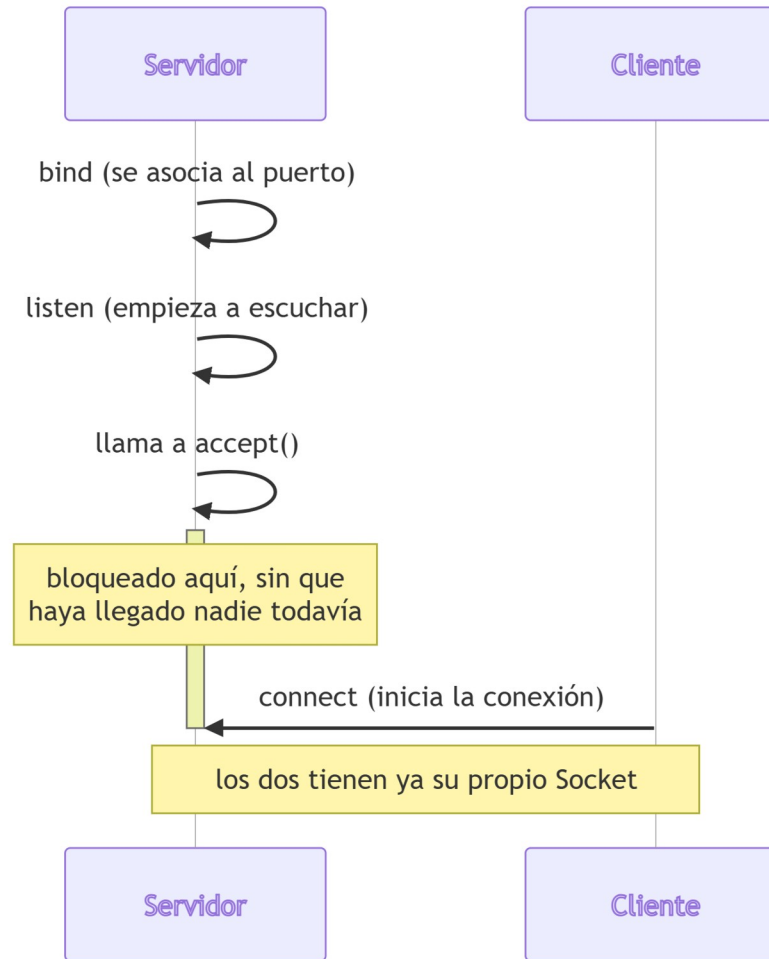
- Servidor: escucha en un puerto concreto, esperando conexiones — como Tomcat en el 8080. En Java clásico, un `ServerSocket`.
- Cliente: inicia la conexión hacia el servidor — como curl, o tu propia aplicación al conectar con PostgreSQL. En Java clásico, un `Socket`.

El servidor y el cliente no hacen lo mismo, ni en el mismo orden — alguien tiene que existir primero y esperar, y alguien tiene que decidir el momento de conectarse.

bind · listen · accept · connect

Paso	Qué hace	Como si...
bind	El servidor se asocia a un puerto — lo reclama para sí mismo	...te mudaras a un piso y pusieras tu nombre en el buzón
listen	Pasa a aceptar que llamen a esa puerta	...abrieras la puerta, listo para recibir visita
accept	Se queda bloqueado hasta que llega alguien; abre un canal dedicado a ese cliente	...te quedaras junto a la puerta hasta que alguien llamara
connect	El cliente inicia la conexión hacia la IP y el puerto del servidor	...fueras tú a la dirección de otra persona y llamaras al timbre

La secuencia completa, en orden



El ServerSocket nunca habla con nadie

Su único trabajo es bind + listen + accept. Cada vez que accept() devuelve, se crea un Socket nuevo y distinto, uno por cliente — es con ese Socket con el que de verdad se conversa, nunca con el ServerSocket.

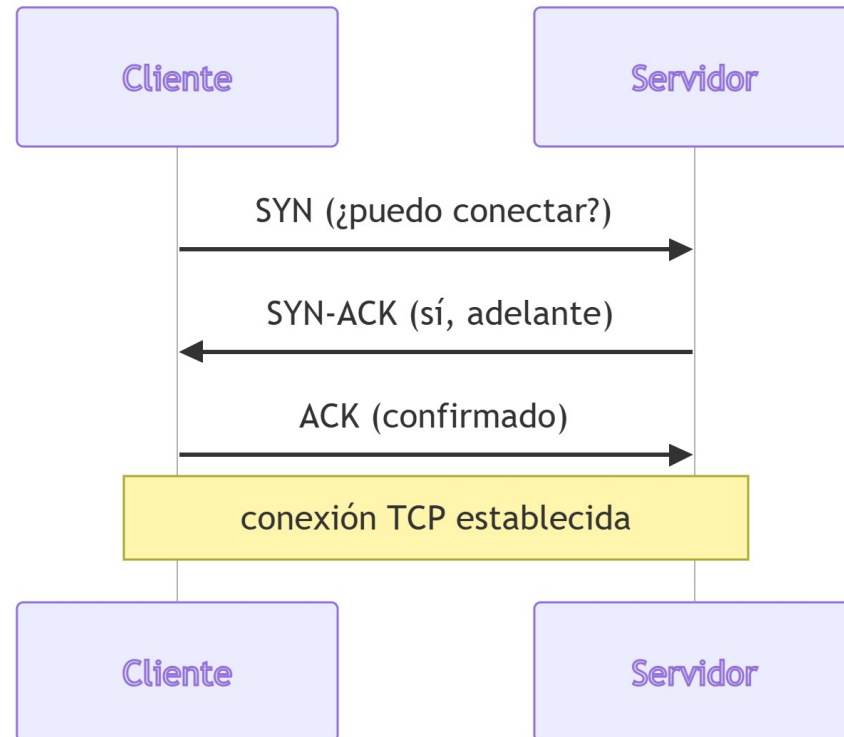
	Servidor	Cliente
Asociarse y escuchar	new ServerSocket(puerto)	—
Aceptar / conectar	serverSocket.accept() (bloqueante)	new Socket(host, puerto)
Leer y escribir	streams sobre el Socket de accept()	streams sobre su propio Socket

accept() es bloqueante y devuelve una sola conexión cada vez: un servidor que solo hace eso no puede atender a un segundo cliente mientras sigue con el primero — lo resuelves con hilos en la Actividad 4.1.

Tipos de socket: TCP vs. UDP

	Stream / TCP	Datagrama / UDP
Conexión	Orientado a conexión — se establece antes de intercambiar datos	Sin conexión — cada paquete viaja independiente
Fiabilidad	Garantiza entrega y orden	Sin garantías
Clases Java	Socket, ServerSocket	DatagramSocket, DatagramPacket
Dónde se usa	HTTP, BD, RabbitMQ — todo lo construido en el curso	Streaming/juegos en tiempo real

"Orientado a conexión": el three-way handshake



Tu código Java nunca escribe estos tres mensajes: los intercambia el sistema operativo. `new Socket(host, puerto)` espera a que el SO complete SYN -> SYN-ACK -> ACK antes de devolver el control.

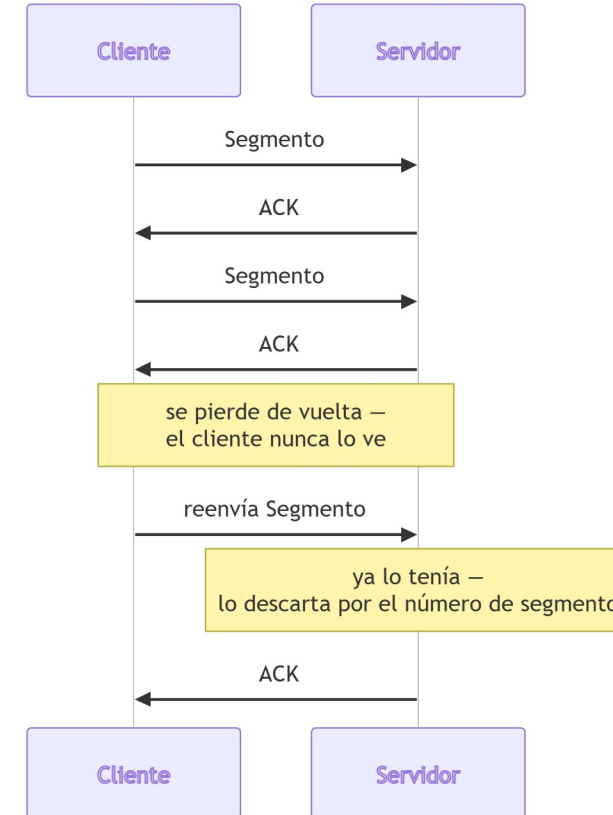
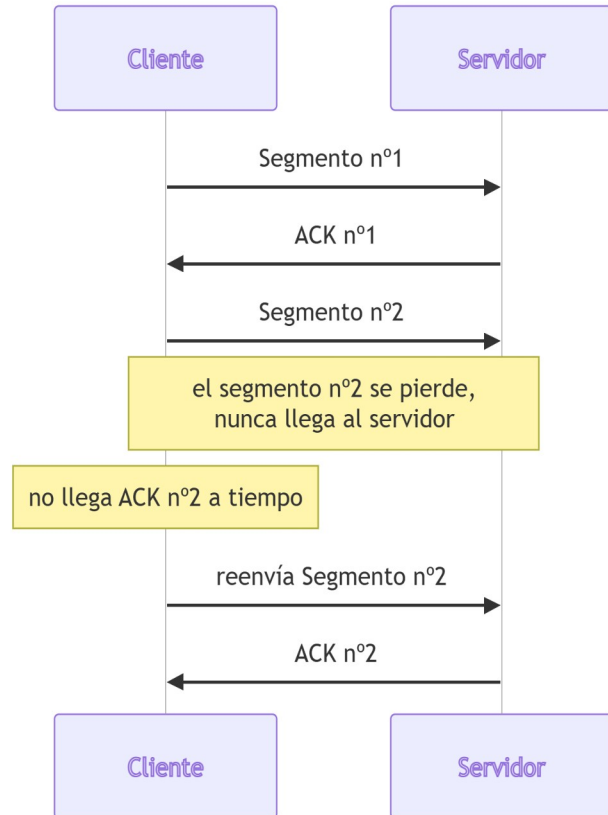
Entrega fiable: segmentos, ACK y retransmisión

TCP divide los datos en segmentos y usa números de secuencia y confirmaciones ACK para saber qué ha llegado. Los ACK pueden confirmar varios segmentos a la vez.

Si el emisor no recibe a tiempo la confirmación, TCP retransmite — puede ser porque se perdieron los datos, o porque se perdió el propio ACK de vuelta.

Tu código nunca ve nada de este ir y venir: ni el reenvío, ni el duplicado descartado. Solo lees de un stream, y los datos llegan completos y en orden.

Dos escenarios de pérdida, resueltos igual



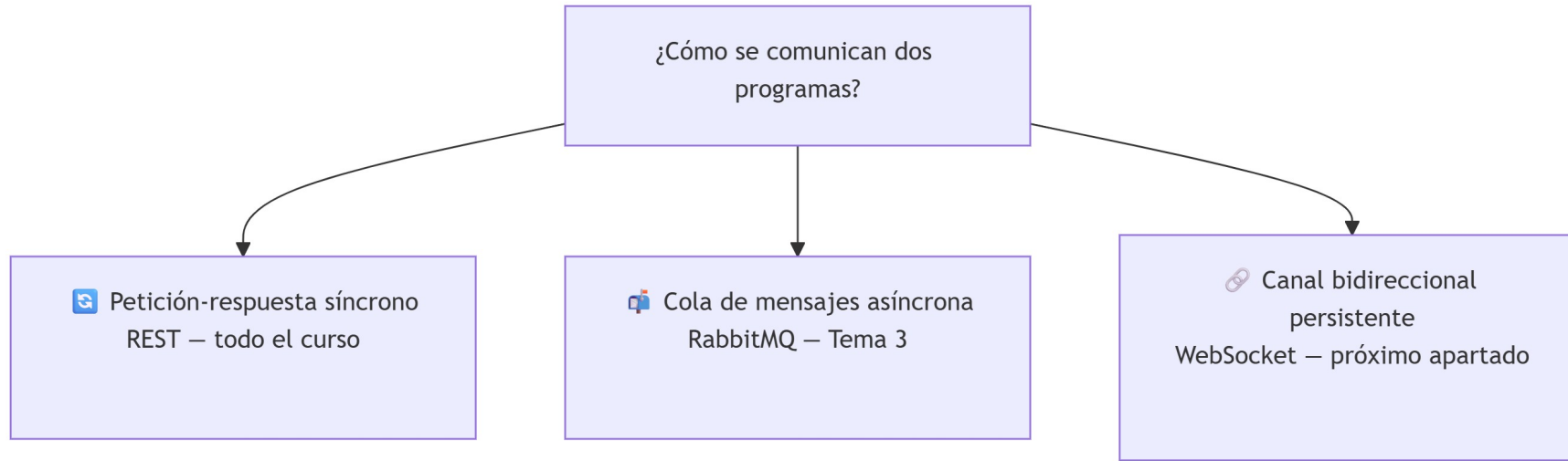
Se pierde el segmento (izquierda) o se pierde el ACK (derecha): en ambos casos TCP reenvía, y los números de secuencia evitan que el servidor entregue el dato duplicado dos veces.

Por qué UDP sigue existiendo

	TCP	UDP
Antes de enviar datos	negocia la conexión primero	envía directo, sin avisar
Si un paquete se pierde	se reenvía solo, tú no te enteras	se pierde y nadie te avisa
Coste del protocolo	más control, confirmación y retransmisión	menos mecanismos de control

En una videollamada, un dato tarde ya no sirve: mejor perder un fotograma que congelarse. En una transferencia bancaria, perder un byte no es aceptable: ahí es TCP quien tiene sentido.

Modelos de comunicación en arquitecturas distribuidas



- Petición-respuesta síncrono: REST — todo el curso.
- Cola de mensajes asíncrona: RabbitMQ (Tema 3) — dentro de una misma cola, cada mensaje lo procesa uno de sus consumidores.
- Canal bidireccional persistente: WebSocket, próximo apartado.

Cuándo no basta con petición-respuesta

Cuando el servidor necesita avisar al cliente de algo que acaba de pasar, sin que el cliente pregunte primero. Con REST puro, el cliente tendría que preguntar constantemente — ineficiente y con retraso.

El caso concreto de las próximas actividades: un panel de actividad en vivo, donde el servidor empuja cada evento del catálogo hacia los clientes conectados, en el instante en que ocurre.

Ideas clave

- Un socket es el extremo programable de una conexión de red (IP + puerto) — está debajo de toda comunicación que ya has usado en el curso.
- Servidor (ServerSocket: bind/listen/accept) escucha; cliente (Socket: connect) inicia la conexión.
- TCP (orientado a conexión, fiable) es lo que usa todo lo construido en el curso; UDP (sin conexión, sin garantías) se usa donde la velocidad importa más que la fiabilidad total.
- Tres modelos de comunicación: petición-respuesta síncrono (REST), cola de mensajes asíncrona (RabbitMQ) y canal bidireccional persistente (WebSocket).
- Un servidor necesita avisar proactivamente al cliente cuando petición-respuesta no basta — el escenario que resuelve el panel de actividad en vivo de las próximas actividades.